CSC3206

Artificial Intelligence

Group Assignment

Assignment 2- Search (Concepts & Implementation)

By:

Group 12

Archie Vun Wei Kian (23041064)

Chia Pei Qin (23015860)

Chua Kay Chun (23046782)

Isabelle Yau Li Shuang（23021165）

Leow Yan Yi（23020324）

Peggy Liow En Ki（23020852）

Department of Smart Computing and Cyber Resilience

School of Engineering and Technology

Sunway University

Table of Contents

Contents	**Page Number**
1.0 Introduction	3
2.0 Implementation Explanation	4-7
3.0 Results Display / Visualization	7-8
4.0 Evaluations and Discussions	8-10
5.0 Conclusion	10
6.0 References	11

1.0 Introduction

In this assignment, we simulate a hex-grid treasure hunt game in which the player must explore the board to gather every available treasure while avoiding traps and obstacles. The player's movement is influenced by game mechanics like the change of gravity and speed, forced player displacement, or removal of the goal.

To smoothly navigate through the board, the A* search algorithm is used to reduce steps and avoid the traps and obstacles. The A* algorithm is a widely used graph traversal and path finding algorithm that determines the shortest path between the beginning node and the goal node in a weighted graph. It combines the best aspects of the Dijkstra's algorithm and Greedy Best-First Search. This combination helps the A* algorithm to make informed decisions on upcoming path explorations (DataCamp, 2024).

A* algorithm is suitable for this scenario because of its optimality, efficiency, and flexibility. When using an admissible heuristic, the A* algorithm guarantees to find the shortest path to the target. Its heuristic-driven approach allows it to outperform uninformed search algorithms by focusing on its search towards the goal. Besides, the A* algorithm is highly adaptable and can be customised through its cost function and heuristic, making it applicable across a wide variety of problem domains (aismartclass, 2024).

There are four assumptions in the implementation. (1) Offset coordinates are used during hexagonal movement. The board uses even-q vertical layouts, which simplifies the data storage and visualisation using 2D arrays while allowing accurate directional movement across six adjacent cells (Red Blob Games, 2013). (2) Traps are always active. The player is immediately and automatically affected when they enter a trap cell. The player's physical position on the board immediately triggers trap effects, ensuring a constant and predictable gameplay experience. (3) Reward cells are one-time use. Each reward is only granted once when the player activates it. Subsequent visits to the same reward cell will not be re-rewarded the same bonus. (4) Visited cells turn beige for visual tracking. For visualisation purposes, cells visited by the player will turn into the colour beige to indicate that they have been traversed. This enables clear visualisation of taken paths and enhances the clarity for debugging and algorithm behaviour demonstration.

2.0 Implementation explanation

This program is designed to collect all treasures using the most efficient path, with the minimum total steps taken and minimum total energy used, while avoiding all traps and obstacles. Players can utilize rewards to ease their journey. The transition model is implemented through the getNeighbors function, which allows movement to any adjacent cell in six possible directions, excluding those that are obstacles. When the program detects that the neighbor cell is an obstacle, it skips that cell which prevents the A* algorithm from considering it as part of the valid path. The getNeighbors function also filters out traps, which helps A* algorithm to avoid traps completely during path planning.

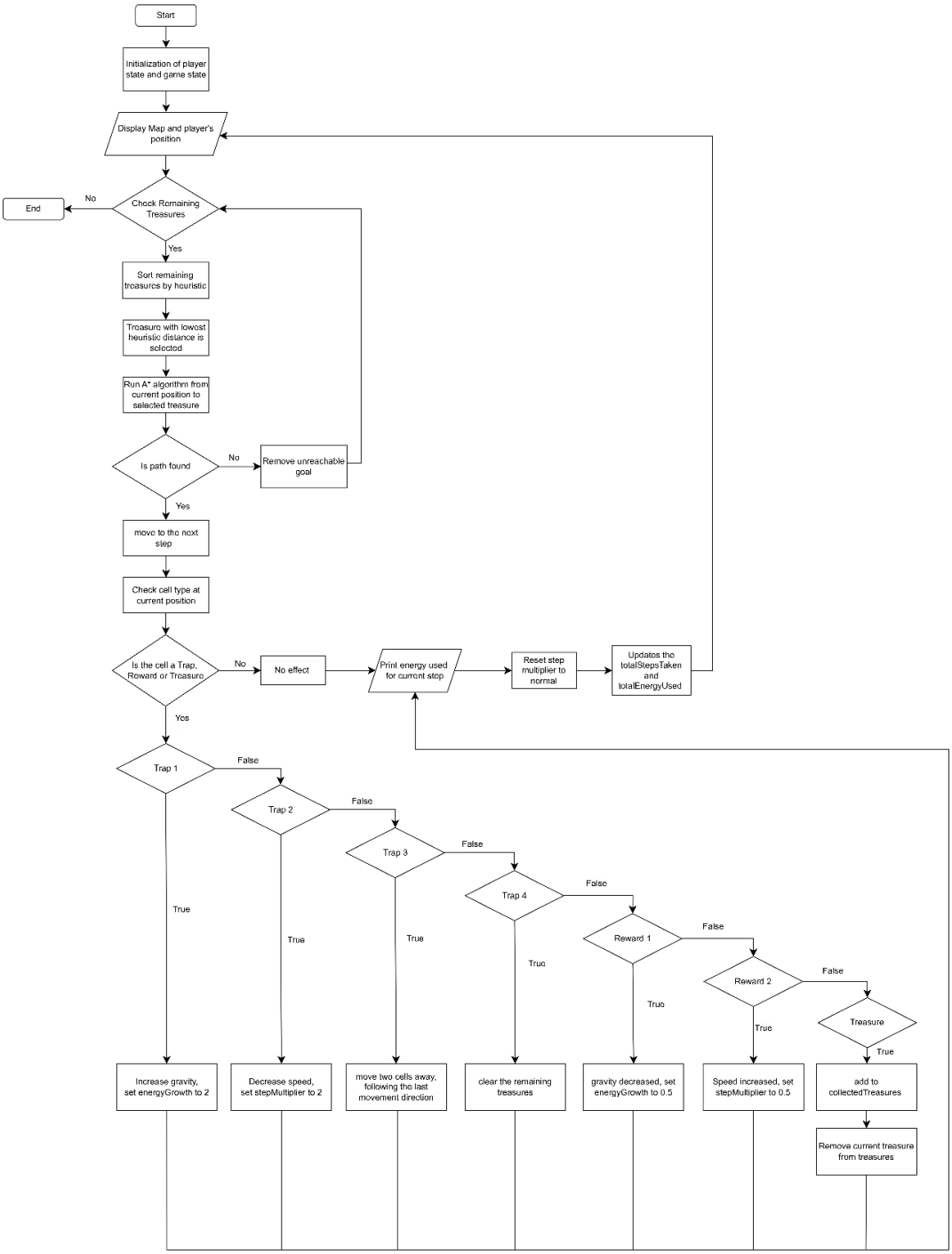


Diagram 2.0.1: Flowchart of Program implementation

The above diagram 2.0.1 shows the flowchart of program implementation. The initial state of the program begins by initializing the player state (pState) and game state (gState). The player state (pState) includes the player's starting position at coordinate (0,0), energy growth rate (energyGrowth =1), step multiplier (stepMultiplier=1) and a set to track collected treasures. These values may change dynamically as the player progresses through the game and interacts with traps or rewards. The player state describes the player's current position, the amount energy used per step, whether they are affected by traps or rewards and the treasures they have collected. Besides, all uncollected treasures found in the map are initially stored in gState['treasures'], while the collectedTreasures set starts empty. This setup allows the program to check the progress and determine when all treasures have been collected.

After the initialization of player state and game state, the program displays the map of the treasure hunt maze including the player's current position. The program then proceeds to check whether there are remaining treasures. If there are no treasures left, the program will end. Otherwise, the program proceeds to sort the remaining treasures by heuristic, which is the estimated distance from the player's current position to each treasure. The treasure with the lowest heuristic distance is selected as the goal to minimize the path length and energy usage.

Next, the program runs the A* algorithm from current position to the selected treasure. During this process, the path cost function is used to calculate the movement cost between neighboring cells. The moveCost function guides the A* algorithm through prioritizing paths that avoids traps and obstacles, while favoring those that utilize rewards. It assigns different costs based on cell types. The empty cells and treasures have a base cost of 1, whereas reward cells have a reduced cost of 0.8 to encourage A* algorithm to select them. Traps have a high cost of 100 to discourage A* algorithm from choosing them. The obstacles cost is set to infinity so that they are entirely excluded from the neighbor search. If the path to the selected treasure is not found, the program removes the unreachable goal and checks whether any treasures remain. Otherwise, the player moves to the next neighboring node and checks the cell type at the current position.

The program checks whether the current cell is a trap, reward or treasure. If the cell does not match any of these, no effect occurs, and the program prints the energy used for the current step. If the cell is Trap 1, energyGrowth is increased to 2 to simulate increased gravity. For Trap 2, stepMultiplier is increased to 2, this is to double the step cost due to reduced speed. For Trap 3, it forces the player to move two cells away in the direction of the last movement. For Trap 4, all the uncollected treasures are removed from the map immediately. On the other hand, Reward 1 sets the energyGrowth to 0.5, simulating the reduced gravity. Reward 2 will set the stepMultiplier to 0.5 to increase the movement speed. If the cell is a treasure, it is added to collectedTreasures and are removed from gState['treasures'], reducing the number of remaining treasures. After the effects of traps, rewards or treasures have been applied, the program prints the energy used for the current step and resets the step multiplier back to its

normal value. The program then updates the totalStepsTaken and totalEnergyUsed based on the current move. Finally, the program displays the map with the latest player's updated position and proceeds to check whether any treasures remain. After all the treasures are collected, the goal state is reached, thus the program ends.

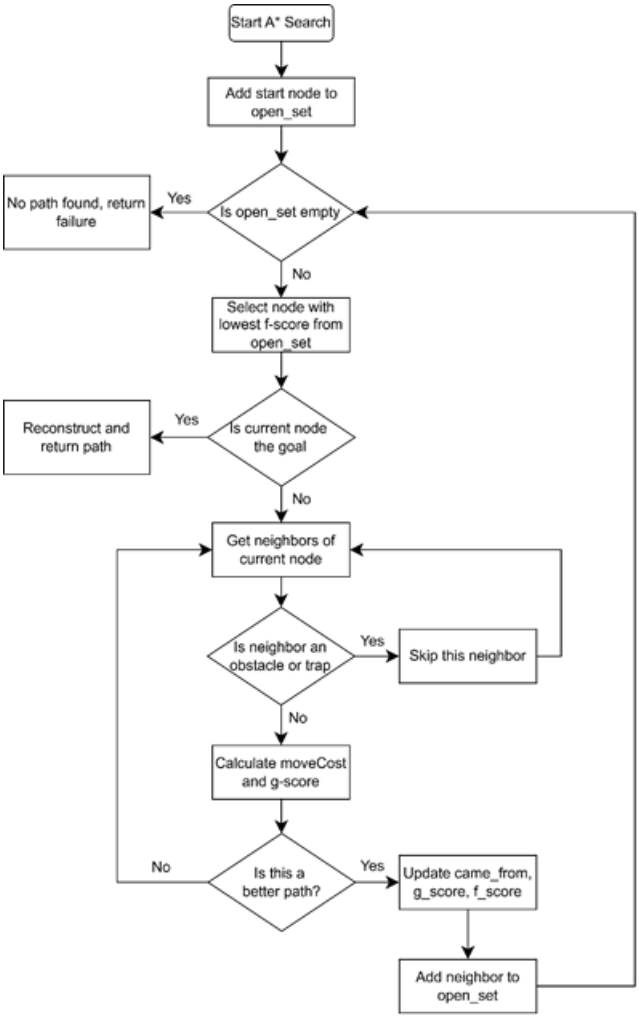
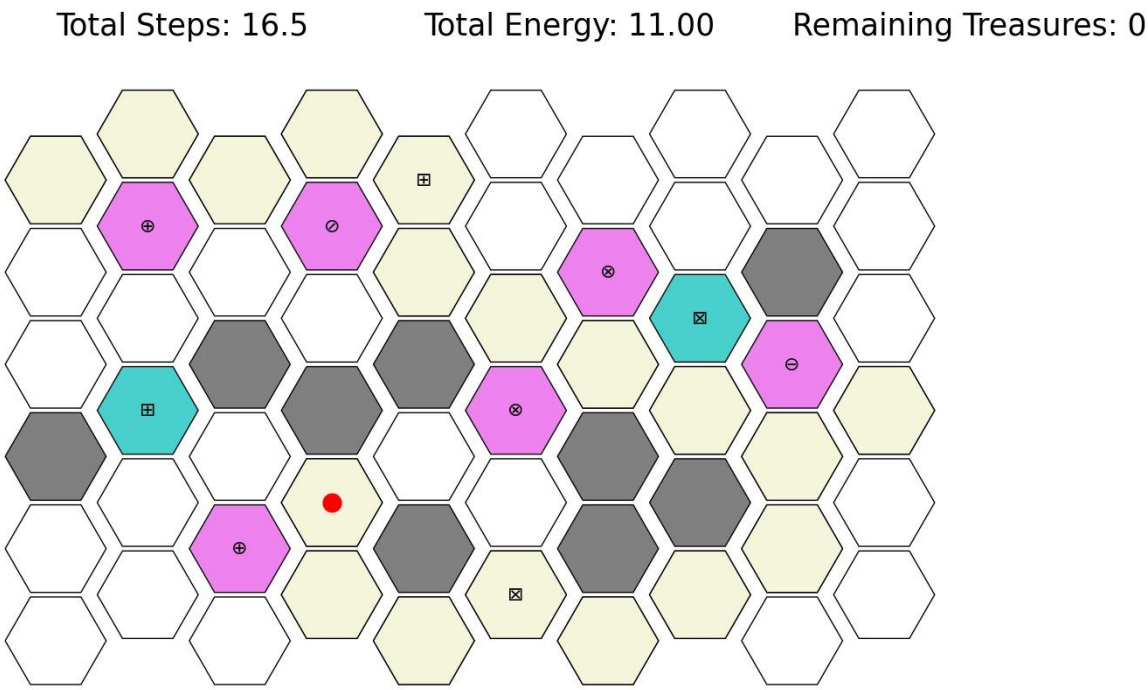


Diagram 2.0.2: Flowchart of A* Search Algorithm Implementation

The above Diagram 2.0.2 shows how the A* Search Algorithm is implemented in our program. The A* algorithm begins when the program needs to find a path to a selected treasure. The algorithm first adds the start node to the open_set, which is a priority queue that holds all possible nodes to explore, prioritized by estimated total cost (f_score). The algorithm then determines whether the open_set is empty. If it is empty, this indicates that no path is found and the search ends. Otherwise, the algorithm selects the node with the lowest f_score from the open_Set. A* algorithm alwas expands the node with the lowest estimated total cost. The algorithm then checks whether the current node is the goal node (treasure). If the current node is the goal node, the algorithm reconstructs and returns the path. Otherwise, the algorithm uses the transition model to find all reachable neighboring cells and checks whether each neighbor is an obstacle or trap. If the neighbor cell is an obstacle or trap, the algorithm skips this cell and proceeds to find other adjacent cells. Otherwise, the algorithm calculates the moveCost and the

tentative g-score, which is the actual cost from the start node to this neighbor. It then checks whether this path is shorter than any previously found path to the same node. If not, it proceeds to explore other neighbors. If the path is better, the algorithm updates the came_from, g_score and f_score and adds the neighbor to the open_set, so that it can be explored in future iterations. The algorithm then loops back to determine whether the open_set is empty and continues the search until the goal is reached.

3.0 Results display / visualisation



By utilising the A* algorithm, the player has successfully gained all four treasures in the map. The beige cells which indicate visited cells tells the algorithm has effectively avoided all traps and obstacles. The final destination is indicated using a red dot, tells that the player has successfully found the optimal path to reach each all treasures. This proves the point that the algorithm is most optimal when paired with a consistent heuristic. Two reward cells were activated in this implementation, allowing the player to enhance its movement efficiency by decreasing the gravity (halved energy growth) and increasing of speed (halved step count). These advantages enabled the player to complete the hunt with minimal steps and energy.

During implementation, no traps were triggered. This is due to the algorithm's nature to always find the lowest-cost path. For example, while approaching cell (0,4) (fourth cell from the left of the first row), the algorithm is aware that the adjacent trap cell would lead to higher actual movement costs after activation, hence it avoids the cell and chose a safer path instead. The overall path resulted in 16.5 steps

and 11 units of energy usage. This shows that the A* algorithm is appropriate for this treasure hunt due to its ability to adapt well to the environment with complex landscape and dynamic elements.

4.0 Evaluation and discussions

We have successfully found the most efficient way to accomplish the goal of collecting every treasure in this treasure hunt maze by applying the A* search algorithm. The algorithm's successful pathfinding capabilities to manage such a dynamic and challenging environment are further demonstrated by its implementation. In the following section, we will reflect on the performance of the A* algorithm by pointing out its advantages and identifying its areas for improvement.

This program combines A* pathfinding with dynamic game mechanics to guide the player to the most efficient and safe path to collect all treasures. First and foremost, the "getNeighbors" function helps the A* search algorithm to find out routes that are viable and low risk by filtering out all obstacles and traps. Besides, the "moveCost" function supports the algorithm to prioritise reward cells and avoid traps. Since the state of the player needs to be updated based on the effect of traps and rewards, it will increase the complexity and realism of path planning. Hence, the A* algorithm is applied continuously to the remaining treasure to allow the player to always pursue the nearest reachable goal to achieve minimal steps and energy.

The A* search algorithm is chosen for this maze due to its ability to combine actual path cost (g(n)) and heuristic estimate (h(n)) to search for the best path. The heuristic is suitable for our hexagonal grid structure since it is based on the cube coordinate distance instead of Euclidean or Manhattan distance. Yang, Li, and Tao (2017) mentioned that this method enhances the heuristic accuracy and ensures the admissibility and consistency of the search output. Hence, using this heuristic enhances the accuracy of the estimated distance between the current location of the player and the treasure target.

Besides, one of the key strengths of implementing the A* search algorithm in this project is the cost function formulation. The cost is dynamically modified depending on the different types of cells in the maze rather than being fixed for each step. In this case, multiple settings have been placed. For instance, Trap 1 and Trap 2 will increase the actual cost of movement while Reward 1 and 2 will reduce the actual cost of the player's movement. Hence, a safer and more efficient path will be chosen by utilizing this system. It will help to evade traps unless there are no available alternatives left. This approach enhances the basic A* algorithm by allowing it to indirectly account for environmental effects, which A* typically does not handle on its own.

Other than that, there is a loop within the algorithm. By exploiting the loop in the algorithm, it can perform search repeatedly and collect all the closest treasures that still exists. Therefore, this strategy ensures all of the treasures will be collected in an easier way. It may also help to avoid the traps

that possess high penalties and enhance the local pathfinding accuracy when combined with the cost function.

Furthermore, the displaymap() function is a function that reflects understandable and significant visual feedback to us. It shows a variety of information, including the current position of the player, visited cells previously, taken and remaining treasures, total steps, and energy used. We can verify the algorithm in different background behaviours based on this visualization information. It also helps us to confirm that the algorithm appropriately responds to traps and rewards when they are triggered. In particular, we can recognize the reduced step cost before it reverts to normal while the player steps on Reward 2.

On the other side, there are various limitations that may influence overall performance still exist on the A* algorithm of the current implementation. First and foremost, Trap 3 may operate accurately when the direction is provided. There will be a possibility that the direction is not passed correctly to the function. In this case, non-functioning forced movement may result in it due to this situation. Thus, inconsistent situations may exist during gameplay. This creates a dependency on external variables that might not always be updated correctly, which could cause the game to behave unexpectedly or miss certain trap effects.

Furthermore, the future effects in terms of trap or reward activations may not be considered in the path planning of the algorithm although it prevents trap cells based on the high movement costs. For instance, the increasing energy cost will not be indicated in the current moveCost calculation while the player steps on Trap 1. The energy cost should have doubled for their future steps. Hence, a path that has the lowest total cost may be chosen based on the algorithm in the scenario, but it might lead to inefficiency because of the high energy cost due to trap activations. According to this limitation, the algorithm might produce different paths or outputs depending on how much of the future it considers, which leads to inconsistencies in decision-making or outcomes.

Last but not least, Perelman (2015) mentioned that the path chosen to collect all treasures may not result in the minimum total steps or energy consumed while using the algorithm. This is because the algorithm selects the closest treasure based on heuristic distance, prioritizing the nearest one from the current location. It may cause the player to collect treasure through an ineffective path, although it may help to reduce time complexity and make real-time decisions effectively. For example, a farther route with better conditions, such as lower penalty and more rewards may appear as treasure. By utilizing the algorithm, this path may be ignored, a nearer treasure that might have lower path costs and higher energy costs due to traps may be chosen. Hence, the current system may not optimize for the overall treasure collection sequence.

In concisely, although the A* search algorithm operates efficiently and adapts the rule of maze well, it still requires various improvements, including incorporating lookahead strategies, modelling

future trap effects, and optimizing treasure collection order. These improvements may help the algorithm to become more intelligent and effective in the future.

5.0 Conclusion

In a nutshell, the application of the A* search algorithm has been demonstrated successfully in this assignment with a virtual treasure hunt maze that contains dynamic elements, including traps, rewards, and obstacles. The goal of this assignment is to use the most efficient path with less energy and steps to collect all treasures. Hence, the hexagonal grid has been navigated intelligently, with prioritizing paths that are safe and effective by combining the A* search algorithm with adaptive cost calculations and the effects of rules.

The implementation ensures the best and most accurate search behaviour by utilizing the cube-coordinate heuristics that are suitable for hex-grid structures. In the rules provided, the movement has been restricted to six directions while the cells filled with traps or obstacles were excluded. Hence, the cost function may used to enhance the ability in terms of adapting to the treasure hunting of the algorithm based on dynamically adjusted path costs based on cell types. This algorithm also managed the player's temporary changes, including energy growth, step multiplier, and the effects of the activation of trap and reward.

Although this algorithm exists with lots of strengths, it also possesses several limitations. Effects that may reflect in the long run will not be considered while using this algorithm, such as Trap 1. Hence, it may increase the energy costs hereafter and not optimize the overall route for all treasures. Besides, the correct direction may need to be put into Trap 3 to activate properly, or else may lead to an inconsistent situation.

In general, this project reflects the performance of the A* search algorithm effectively in an environment with dynamic and rule-based. This algorithm may become more strategic and effective in the future by enhancing to consider future considerations and global path optimization.

6.0 References

aismartclass. (2024, April). *What is (A*) A-Star Algorithm? Empowered: With 3 Advantages and Disadvantages*. https://aismartclass.net/blog/what-is-a-star-algorithm/#advantages-of-a-star-algorithm

Kumar, R. (2024, November). *The A* Algorithm: A Complete Guide.* DataCamp. https://www.datacamp.com/tutorial/a-star-algorithm

Perelman, B. S. (2015). *A naturalistic computational model of human behavior in navigation and search tasks* (Doctoral dissertation, Michigan Technological University). https://doi.org/10.37099/mtu.dc.etds/935

Red Blob Games. (2013, March). *Hexagonal Grids*. https://www.redblobgames.com/grids/hexagons/

Yang, R., Li, C., & Tao, Z. (2017). *Realization of maneuver trajectory in computer wargaming system based on an improved A* algorithm*. In *Advances in Engineering Research (Vol. 102)*. Atlantis Press. https://doi.org/10.2991/icmmse-17.2017.67